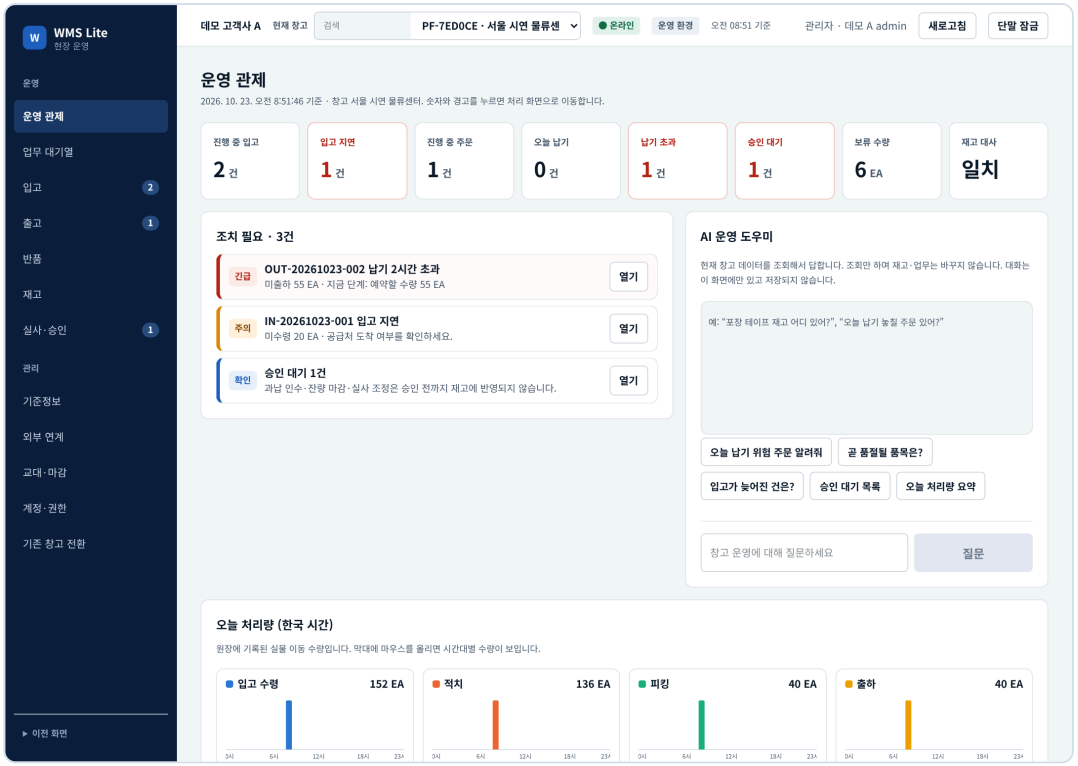
PORTFOLIO 고객사별 창고 운영관리 SaaS

WMS Lite

입고부터 출하·반품까지, 실물·잔량·담당자를
서버 규칙과 지울 수 없는 원장으로 연결한 웹 창고 운영 시스템

Next.js 16 · React 19 · TypeScript · PostgreSQL · RLS · Better Auth · Vercel + Neon · Playwright · Vitest · Claude API

작성자 **전우석**
개발 기간 **2일 (기획·규칙 결정·검증 주도, AI 도구로 구현)**
형태 **1인 프로젝트 · 공개 저장소 · 공개 데모**



운영 관제 화면 · 로컬 실행 · 합성 데이터

02 프로젝트 개요

여러 고객사가 한 서비스에서 창고를 운영합니다

"재고가 있다"와 "출고할 수 있다"를 구분하고, 모든 수량 변화를 누가·왜 했는지까지 남기는 것이 목표입니다.

무엇을 하는 시스템인가

회사(고객사)마다 창고·위치·품목·거래처를 등록하고, **입고 → 검수 → 적치**, **주문 → 예약 → 피킹 → 포장 → 인계·배송**, **반품·실사·승인·교대 마감**을 처리합니다. 외부 시스템은 연계 API로 주문·입고 사건을 보내고, 관리자는 운영 관제 화면과 읽기 전용 AI 도우미로 위험을 찾습니다.

핵심 규칙 3개

1. **재고는 덮어쓰지 않는다**
   모든 증감은 UPDATE·DELETE가 막힌 원장에 한 줄씩 쌓고, 현재고가 원장 합계와 같을 때만 다음 명령을 실행합니다.

2. **같은 요청은 한 번만 반영한다**
   화면이 만든 request_id로 응답이 끊긴 재시도도 같은 결과를 돌려주고, 내용이 다르면 거절합니다.

3. **권한 경계와 승인 분리**
   회사·창고·역할을 서버와 DB(RLS)가 이중으로 검사하고, 과납·실사 차이·정정은 요청자와 다른 사람이 승인합니다.

규모

41 DB 테이블 SQL 마이그레이션 22개	**40** 업무 명령 receipt.receive 등	**51** API 라우트 사람·기계 키 공용
19 화면 현장 운영 12 · 기존 7	**696** 자동화 테스트 단위·통합·브라우저 등	**7** 재고 지표 H·S·A·R·P·Q·B

재고 7지표: 총실물 H = 정상보관 S + 피킹 P + 검수·적치 Q + 보류 B. 예약 R은 S 안에 포함되고, 가용 A는 예약과 위치 잠금을 뺀 수량입니다.

처리 범위

기준정보·CSV · 입고·검수·적치 · 출고·예약·피킹·포장·배송 · 반품 · 실사·승인 · 교대·일일 마감 · 외부 연계 · 운영 관제·AI · 기존 창고 전환

사용자와 역할

회사 관리자 계정·권한·기준정보·승인 · **운영 책임자** 배정·예외·승인 · **작업자** 수령부터 출하까지 실물 처리 · **조회자** 조회만

03 시스템 아키텍처

업무 규칙과 권한 검사는 모두 서버에 있습니다

화면은 안내만 하고, 수량·상태·권한은 서버와 DB가 다시 검사합니다. 서버 함수와 DB는 같은 싱가포르 지역에 둡니다.



한 앱에 화면과 API

화면은 src/app/dashboard, API는 src/app/api 아래 51개 라우트입니다. 서버 컴포넌트에서 세션을 확인해 로그인 화면으로 보냅니다.

엔진을 DB와 분리

“현재 상태 + 명령 → 새 상태 + 원장 행”만 계산하므로 DB 없이 단위 테스트로 업무 규칙 40개 명령을 검사합니다.

외부 연계도 같은 문

시뮬레이터와 외부 시스템은 회사·창고·범위가 묶인 기계 키로 같은 API와 같은 권한 검사를 거칩니다.

계층마다 “무엇을 막기 위해” 골랐는지 정리했습니다

계층	선택	선택 이유	대안과 대가
화면·API	**Next.js 16 App Router** **React 19**	화면과 API를 한 저장소·한 배포로 관리. 서버 컴포넌트에서 세션을 확인해 페이지 단위 보호	프레임워크 변경이 잦아 버전별 문서를 확인하며 작성
언어	**TypeScript 7**	명령 페이로드·업무 상태 타입을 화면·서버·엔진이 공유해 필드 누락을 컴파일 단계에서 발견	—
DB	**PostgreSQL**	트랜잭션·행 잠금·advisory lock·RLS를 DB가 보장. 동시 예약과 회사 간 노출을 앱 코드만 믿지 않고 막음	회사별 DB 분리보다 격리는 약하지만 운영 비용이 낮음 → RLS FORCE로 보완
스키마	**node-pg-migrate · SQL**	RLS 정책·불변 트리거·인덱스를 SQL로 명시하고 22개 마이그레이션으로 이력 관리	ORM 편의 기능 대신 SQL을 직접 검토
인증	**Better Auth**	세션·초대·비밀번호 재설정을 직접 구현하지 않음. 보호 요청마다 서버에서 유휴 시간 검사	세션 확인에 DB 왕복이 추가됨
배포	**Vercel(sin1) + Neon**	무료 공개 데모. 서버 함수와 DB를 같은 싱가포르 지역에 두어 요청당 DB 왕복 지연 최소화	Neon 무료 요금제는 5분 유휴 시 DB가 잠들어 첫 요청이 느림
AI	**Claude API 도구 호출**	현재 창고 데이터로 답하되 읽기 전용 도구 7개만 제공해 재고 변경 경로를 원천 차단	API 키가 없으면 빠른 질문은 같은 도구를 직접 실행
검증	**Vitest · Playwright · node:test**	단위(엔진·권한)·실제 PostgreSQL 통합·실제 브라우저 흐름을 한 명령(npm run verify)으로 실행	GitHub Actions 대신 로컬 검증을 원칙으로 함
시뮬레이터	**Node 24 · node:sqlite**	WMS 코드를 재사용하지 않는 별도 앱이 API로 업무를 보내고, 독립 기대값으로 재고·원장을 대조	시험 회사 전용, 현장 성능 검증을 대체하지 않음

41개 테이블을 성격에 따라 네 묶음으로 나눴습니다

모든 업무 테이블에 tenant_id가 있고, 회사 경계는 RLS가 강제합니다. 아래는 핵심 테이블과 관계입니다.

기준정보

바뀔 수 있지만 참조 중이면 비활성화 불가

- **tenants**
  회사 · 유휴 잠금 정책
- **warehouses → locations**
  창고 · 위치(수령·보관·보류·포장)
- **items**
  품목 · 최소 재고
- **business_partners → delivery_destinations**
  거래처 · 배송지
- **master_barcodes**
  품목/위치 바코드 · 포장 수량 변경 금지
- **memberships · v2_warehouse_grants**
  회사 역할 · 창고 허가
- **v2_machine_keys**
  외부 연계 키 · 범위

현재 상태

창고 단위로 잠가 갱신

- **v2_workflow_state**
  창고별 업무 상태 헤더
- **v2_workflow_object_state**
  입고예정·주문·포장·실사 등 객체 상태
- **v2_workflow_stock**
  품목×위치의 S·R·P·Q·B 현재고
- **v2_workflow_objects**
  외부번호와 객체 연결
- **v2_integration_entities**
  외부 업무 최신 버전

지울 수 없는 이력

UPDATE·DELETE 금지 트리거

- **v2_workflow_ledger**
  재고 증감 한 줄씩: 품목·위치·버킷·증감·명령·작업자·사유
- **v2_workflow_requests**
  request_id · 요청 해시 · 응답
- **v2_workflow_performances**
  작업 실적 · 스캔 증거
- **v2_workflow_receivings**
  수령 단위 중복 방지
- **api_request_metrics**
  요청별 결과·처리 시간

운영 기록

연계·계정·전환의 처리 기록

- **v2_integration_events**
  외부 사건 · 격리 · 재처리
- **v2_identity_audit**
  계정·권한 변경 이력
- **master_csv_previews / applies**
  CSV 검증 미리보기 · 적용
- **v2_opening_imports · v2_cutovers**
  개시재고 · 기존 창고 전환
- **session · rateLimit**
  세션 · 로그인 시도 제한

v2_workflow_ledger 합계 = v2_workflow_stock
명령 실행 전마다 대사, 다르면 실행 거부

v2_workflow_requests 1:N ledger · performances
같은 request_id로 묶여 함께 COMMIT

items · locations ← stock · ledger
참조 중인 기준정보는 비활성화 거절(트리거)

현재 상태는 “결과”, 원장은 “원인”

현재고는 빠른 조회용이고, 정답은 원장입니다. 둘이 다르면 새 명령을 받지 않고 원인을 찾게 합니다.

이력은 고치지 않고 덧붙인다

원장·요청 결과·작업 실적은 트리거로 변경을 막고, 잘못은 승인을 거친 반대 방향 실적으로 바로잡습니다.

외부 사건은 업무와 분리

연계 사건은 접수와 반영을 나눠 저장해, 순서가 뒤바뀌거나 중복돼도 업무 상태가 오염되지 않습니다.

06 데이터 저장 흐름

“실물 수령 확정” 요청 하나가 한 트랜잭션 안에서 처리되는 순서

POST /api/v2/workflow · executeWorkflowInTransaction · 실패하면 어느 단계든 전체 ROLLBACK

1	**화면: request_id 생성 후 먼저 저장** UUID를 만들어 브라우저에 “결과 미확인”으로 저장하고 전송	응답이 끊겨도 같은 키로 결과 확인
2	**API: 출처·세션·멤버십·창고 권한** 유휴 잠금 검사, 역할·창고 허가를 DB에서 다시 읽음	회사 헤더 위조, 퇴사자 세션
3	**BEGIN + RLS 문맥 설정** set_config(app.tenant_id, app.user_id …)를 BEGIN과 한 번에 전송	다른 회사 행 조회·수정
4	**request_id 잠금** pg_advisory_xact_lock(회사ID:request_id)	같은 요청의 동시 이중 실행
5	**창고 상태 행 잠금 + 대사** FOR UPDATE로 창고 명령을 한 줄로 세우고 원장 합계=현재고 확인	동시 예약 초과, 틀어진 상태 위의 작업
6	**저장된 요청 조회** 같은 키·같은 해시면 이전 응답 반환, 해시가 다르면 IDEMPOTENCY_CONFLICT	재시도로 인한 이중 반영
7	**엔진 실행 (순수 함수)** applyCommand(상태, 명령) → 새 상태 + 원장 행 (예: Q +24)	수량 초과·위치 유형·버전 충돌
8	**원장·현재고·요청 결과 저장 → COMMIT** ledger INSERT, stock 갱신, requests에 해시·응답 저장	일부만 저장되는 상태
9	**화면: 결과 미확인 삭제 → 다음 단계** 응답의 스냅샷으로 화면 갱신, 다음 할 일(검수 24 EA) 안내	—

권한에서 막히면

트랜잭션을 열기 전에 401·403으로 끝나며 DB에 업무 흔적이 남지 않습니다.

엔진에서 막히면

EXCEEDS_PENDING 같은 업무 오류와 함께 전체 ROLLBACK, 원장·현재고 변화가 없습니다.

응답만 사라지면

서버는 이미 COMMIT. 화면이 저장해 둔 같은 request_id로 다시 보내 저장된 결과를 받습니다.

단계 3은 BEGIN과 설정 쿼리를 한 번에 보내 DB 왕복을 줄인 현재 구현입니다.

07 핵심 코드

검수: “정상”은 적치 대기로, “보류”는 원장에 Q→B로 남깁니다

실제 코드 발췌 · src/server/v2/workflow/engine.ts · 고정 커밋 ca8c389

inspect() · L216

```
function inspect(s: WorkflowState, ctx: WorkflowContext, parent: { lots: Lot[] }, p: Record<string, unknown>) {
  for (const x of rows(p.lines)) {
    const lot = find(parent.lots, x.lot_id);
    const q = integer(x.quantity);
    if (lot.pending < q) fail("EXCEEDS_PENDING");
    location(s, ctx, lot.location_id);
    if (x.outcome === "normal") {
      lot.pending -= q;
      lot.normal += q;
    } else if (x.outcome === "hold") {
      lot.pending -= q;
      lot.hold += q;
      if (lot.overage) lot.quality_hold = (lot.quality_hold ?? 0) + q;
      change(s, ctx, lot.item_id, lot.location_id, "Q", -q, p);
      change(s, ctx, lot.item_id, lot.location_id, "B", q, p);
    } else fail("INVALID_OUTCOME", 400);
  }
}
```

change() · L106 (원장 기록부 생략)

```
function change(
  s: WorkflowState,
  ctx: WorkflowContext,
  item: string,
  loc: string,
  bucket: Bucket,
  delta: number,
  p: Record<string, unknown>,
) {
  if (!delta) return;
  if (!ctx.items.some((i) => i.id === item && i.active)) fail("ITEM_UNAVAILABLE");
  const r = stock(s, item, loc);
  r[bucket] += delta;
  if (!Number.isSafeInteger(r[bucket]) || r[bucket] < 0 || r.R > r.S) fail("INSUFFICIENT_STOCK");
  s.ledger.push({ /* 품목·위치·버킷·증감·명령·작업자·사유 */ });
}
```

engine.ts 원문 ↗

① 남은 수량을 넘지 못함

lot.pending(수령했지만 검수 전)보다 많이 검수하면 EXCEEDS_PENDING. 화면 검사가 빠져도 서버에서 거절합니다.

② 정상과 보류의 재고 효과가 다름

정상은 실물이 그대로 수령 위치에 있으므로 Q를 유지하고 적치 단계에서 Q→S로 옮깁니다. 보류는 즉시 Q−q, B+q 두 줄을 원장에 남겨 가용 재고에서 빠집니다.

③ 모든 증감은 change() 한 곳

비활성 품목, 음수 재고, 예약 R이 정상 S를 넘는 상태를 여기서 막고 원장 행을 만듭니다. 재고를 바꾸는 다른 길이 없습니다.

④ DB를 모르는 함수

같은 입력이면 같은 결과라 단위 테스트로 검수·적치·보류 해제를 검사하고, 저장·잠금은 6쪽의 트랜잭션이 맡습니다.

08 보안·권한 구조

회사 → 창고 → 역할 → 승인 자격을 차례로 검사합니다

앞 단계를 통과해도 다음 단계에서 다시 검사합니다. 세션에 저장된 옛 역할은 믿지 않습니다.

1. 인증

Better Auth 세션. 보호 요청마다 회사 정책의 유휴 시간을 서버가 검사해 초과 세션은 삭제합니다. 외부 연계는 회사·창고·범위가 묶인 기계 키(SHA-256 해시 저장)를 씁니다.

2. 회사 경계

회사는 요청 헤더가 아니라 세션 사용자의 멤버십으로 정합니다. DB는 FORCE RLS + NOBYPASSRLS 계정으로, 앱 코드가 조건을 빠뜨려도 다른 회사 행을 돌려주지 않습니다.

3. 창고·역할

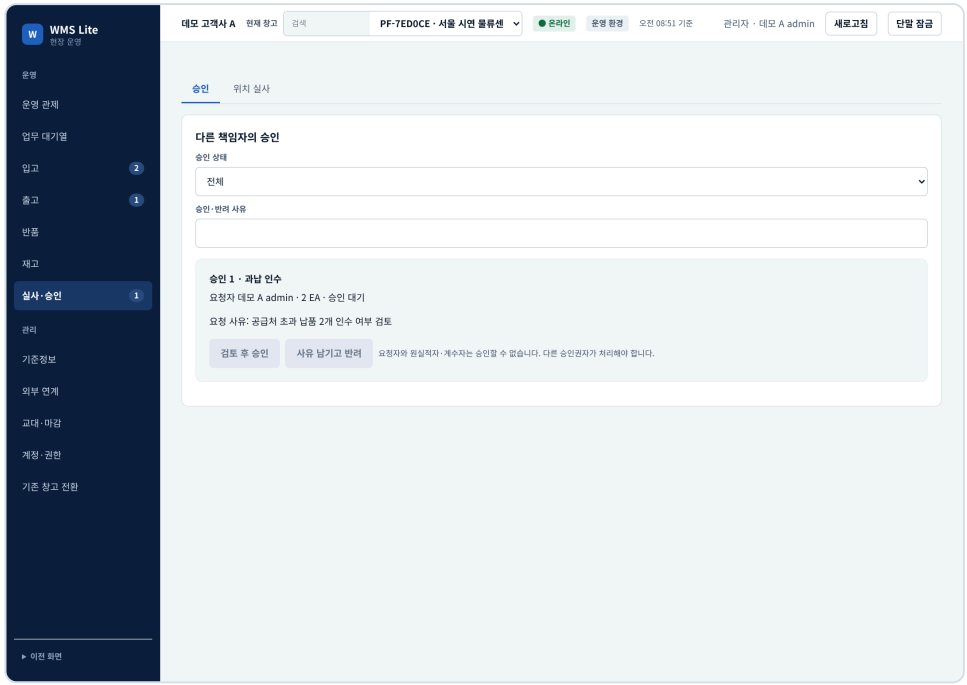
v2_warehouse_grants에 허가된 창고만 조회·처리합니다. 역할표: 관리자(계정·기준정보·승인) · 책임자(배정·예외·승인) · 작업자(실물 처리) · 조회자(조회만).

4. 승인 분리

과납 인수·잔량 마감·실사 차이·정정은 요청자·원실적자·계수자를 제외한 다른 승인권자만 확정합니다. 대상 버전이 바뀌면 APPROVAL_STALE로 무효입니다.

5. AI 경계

도우미 도구는 읽기 전용 7개뿐이고 로그인 사용자의 창고 권한으로만 조회합니다. 도구 결과의 품목명·사유는 지시가 아닌 데이터로 취급합니다.



본인이 요청한 과납 승인: 승인 버튼이 비활성화되고 다른 승인권자 처리를 안내합니다.

검증으로 확인한 것

회사 A↔B 조회·전표 접근·취소 차단과 데이터 미변경, 메뉴 이동 중 세션이 끝나면 “권한 없음” 대신 로그인 화면으로 보내는 동작을 브라우저 시험으로 고정했습니다.

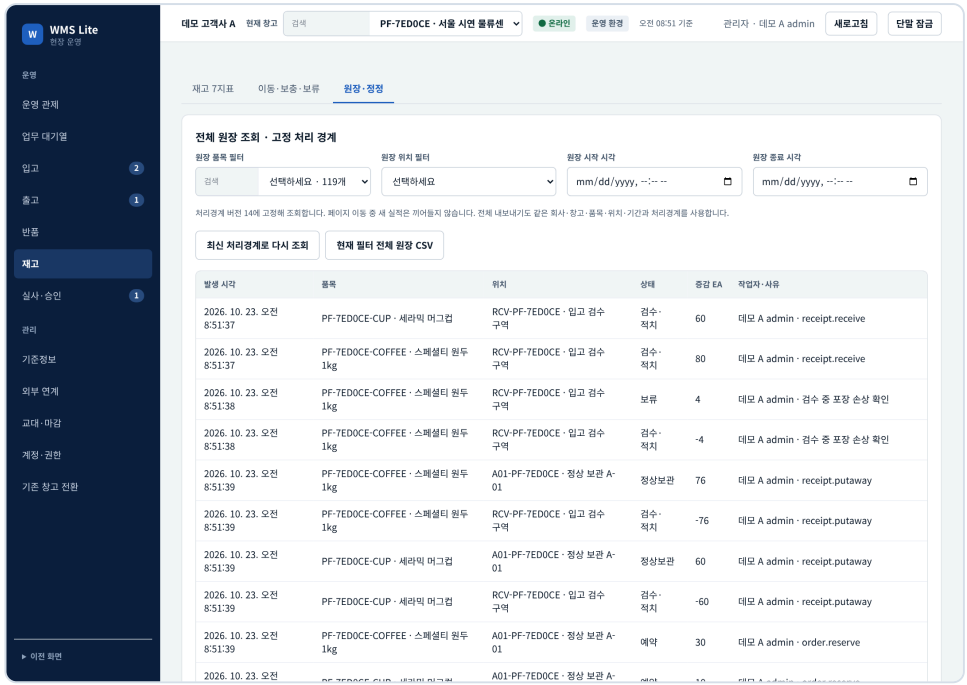
09 동시성과 정합성

동시 작업과 재시도에도 재고와 원장이 어긋나지 않게

창고 단위로 명령을 한 줄로 세우는 대신, 처리량 상한을 감수했습니다.

상황	장치	결과
두 작업자가 마지막 10개를 동시에 예약	창고 상태 행 `FOR UPDATE`	뒤 요청은 앞 요청 결과를 보고 판단 → 초과 예약 없음
확정 응답이 끊겨 다시 누름	`request_id` advisory lock + 요청 해시	저장된 응답 재반환(replayed), 재고 1회 반영
같은 키로 다른 수량 전송	해시 비교	IDEMPOTENCY_CONFLICT 거절
화면이 보던 업무가 그사이 변경	`expected_version` (낙관적 잠금)	VERSION_CONFLICT, 새로 보고 다시 처리
이미 틀어진 재고 위에 작업	명령 전 원장 합계 = 현재고 대사	실행 거부, 원인 조사부터
처리 중 오류	원장·현재고·요청 결과 단일 트랜잭션	전체 ROLLBACK, 일부 저장 없음
잘못 처리한 수량	원장 `UPDATE·DELETE` 금지 트리거	지우지 않고 승인 후 반대 방향 정정 실적

> **대가와 기록** 한 창고의 처리량에 상한이 생깁니다. 혼잡 시험 기록(docs/v2-congestion-*)에 목표 미달 결과를 숨기지 않고 남겼고, 전체 상태 읽기와 잠금 대기를 병목 후보로 정리했습니다.



원장 조회: 품목·위치·처리 경계로 모든 증감과 작업자·사유를 추적합니다.

예약은 “전부 아니면 실패”

부족하면 운영자가 “가능 수량만 예약”을 명시적으로 고릅니다. 시스템이 몰래 일부만 잡지 않습니다.

사람 없이 업무를 흘려 보고, 사람은 설명 없이 쓰게

자동화

가상 데이터 시뮬레이터

Node 24·SQLite의 별도 앱이 기계 키로 연계 API를 호출해 입고·주문·운송 사건과 부족·파손·오스캔을 만듭니다. WMS 계산 코드를 쓰지 않는 독립 기대값(oracle.mjs)으로 재고·원장·원업무 수량을 대조합니다.

외부 연계 API

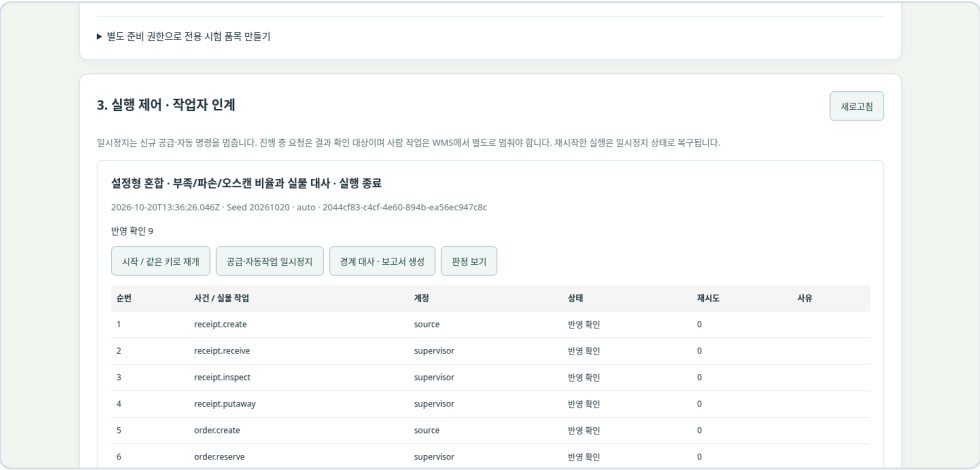
사건마다 버전·멱등 키를 두고 의존성 대기·업무 거절·격리를 구분합니다. 관리자는 원래 사건으로 재처리합니다.

CSV 일괄 등록

기준정보·입고예정·주문을 미리보기로 형식·참조·중복을 검사한 뒤 정상 행만 골라 적용합니다.

운영 관제·AI

원장을 집계해 처리량·납기 위험·품절 예측(최근 7일 60% + 28일 40% 가중)을 계산하고, 읽기 전용 도우미가 같은 데이터로 답합니다.



시뮬레이터 실행 타임라인 (기존 로컬 시험 캡처)

프론트엔드

스캔 우선 작업 화면

위치 → 품목 바코드 순서를 안내하고, 반복 스캔은 포장 수량만큼 누적합니다. 스캔 실패 시 검색 대체는 사유를 남깁니다.

다음 할 일과 막힌 이유

서버가 계산한 “검수할 수량 24 EA” 같은 안내를 보여 주고, 확정 버튼이 비활성이면 이유를 바로 위에 씁니다.

결과 미확인 복구

응답을 못 받은 확정이 있으면 새 작업을 막고 “원래 요청 그대로 재전송”으로 복구합니다.

공용 단말과 속도

무활동 자동 잠금, 마지막 창고를 기억해 로그인 정보와 동시에 불러오기, 기준정보 창고별 재사용, 글꼴 자체 호스팅.



태블릿 폭의 운영 관제

11 업무 흐름과 주요 화면 ①

입고: 예정만으로 재고를 만들지 않습니다

실제로 처리한 수량만 다음 단계로 넘기고, 단계마다 남은 수량을 유지합니다.

입고예정
공급처·품목·예정 수량
재고 변화 없음

→

1 수령
실제 도착 수량
Q +수령

→

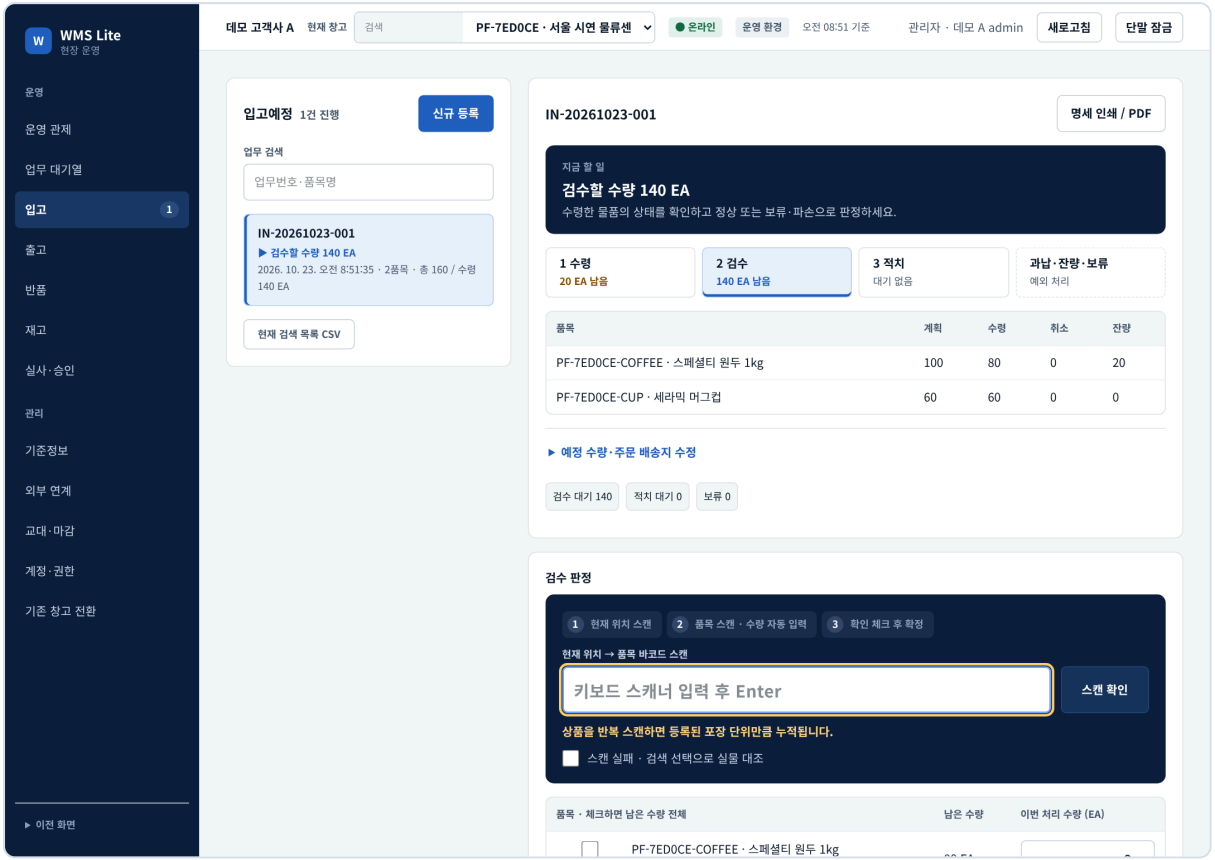
2 검수
정상: Q 유지
보류: Q→B

→

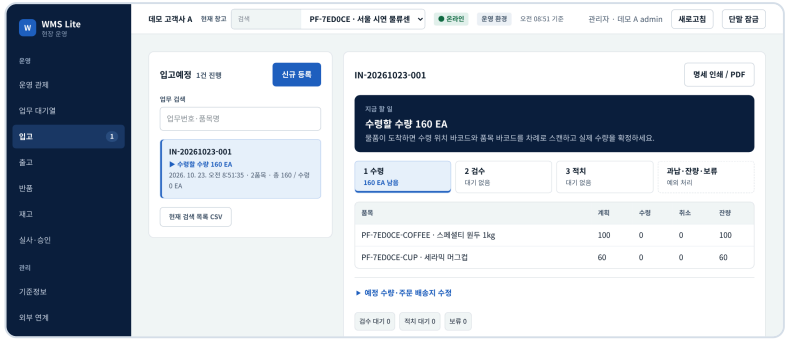
3 적치
보관 위치 확정
Q→S, 가용 증가

…

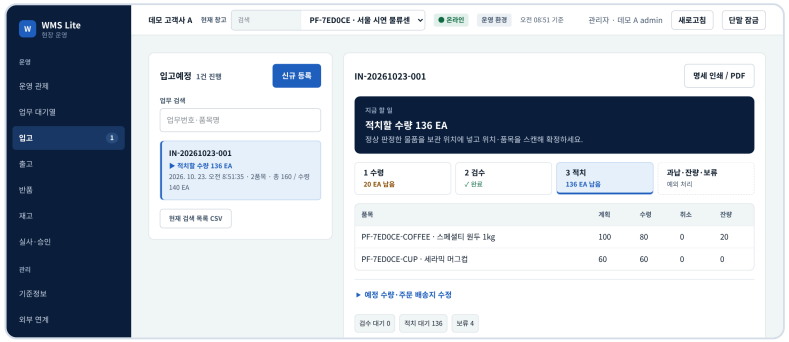
예외
과납은 보류 기록 후 독립 승인
미도착 잔량 마감은 승인



검수: 160 EA 중 140 EA 수령. 미수령 잔량과 검수 대기를 구분합니다.



수령: 위치·품목 스캔 후 실제 수량 확정



적치: 정상분을 보관 위치로

출고: 예약한 곳에서 꺼내 실제 인계 시점에 차감합니다

주문 → 예약 → 피킹 → 포장 → 인계·배송. 단계 이동과 재고 반영 시점을 구분합니다.

주문
거래처·배송지 필수
재고 변화 없음

→

1 예약
가용 확보
R +예약

→

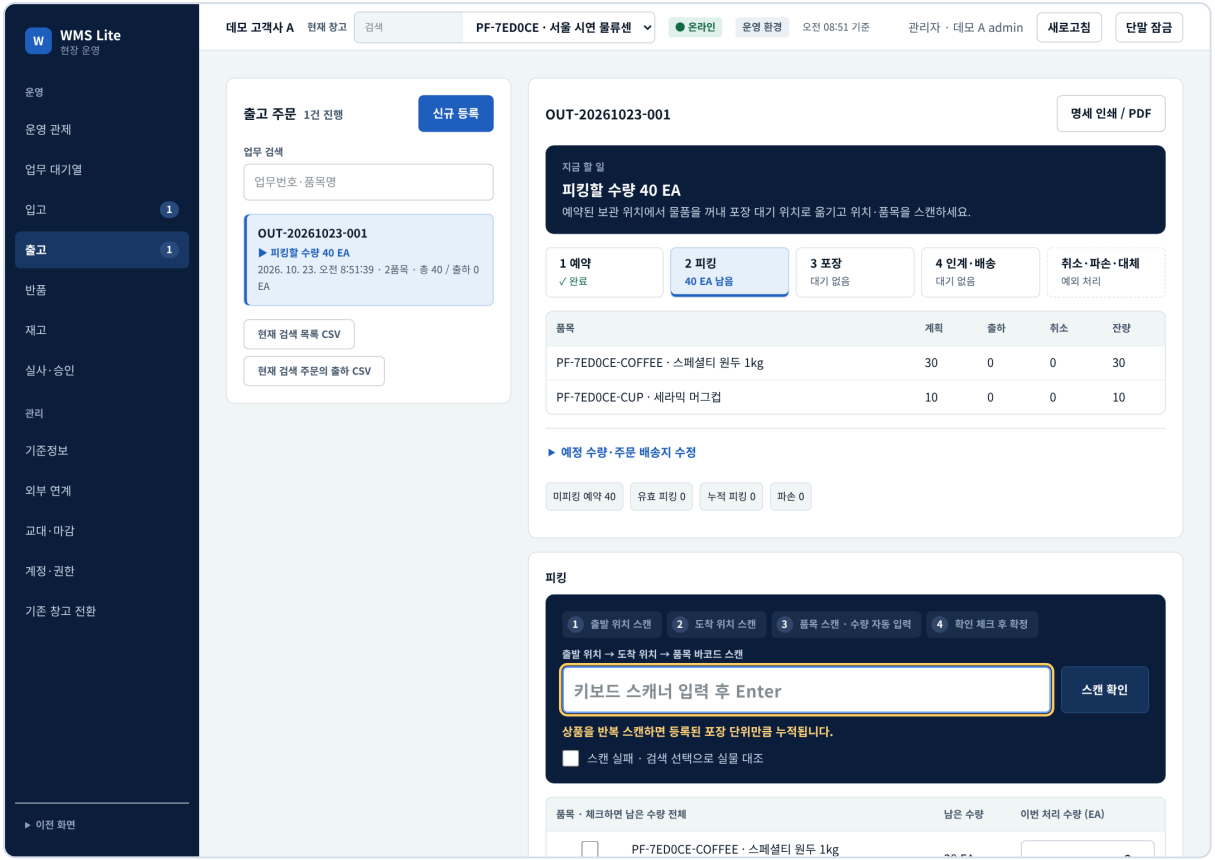
2 피킹
예약 위치에서 꺼냄
S·R → P

→

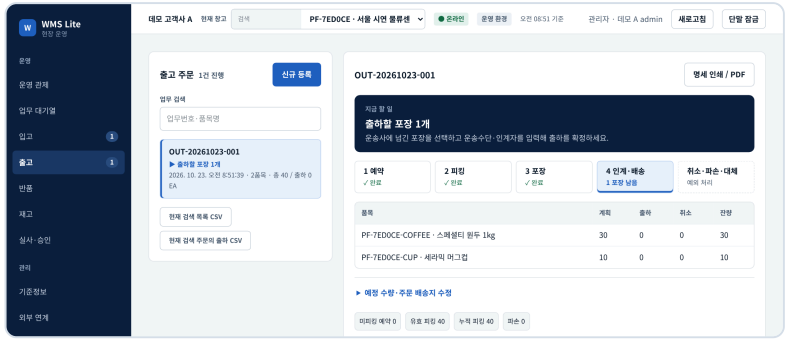
3 포장
포장 단위 구성
배송지 스냅샷

→

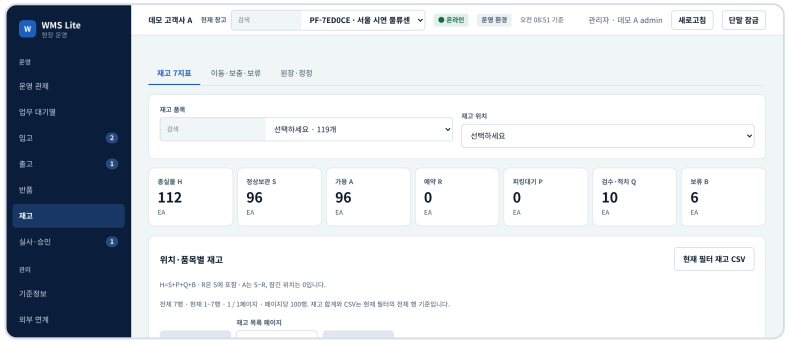
4 인계·배송
실제 출하
P 차감



피킹: 예약 40 EA 후 지금 할 일과 단계별 잔량을 안내합니다.



인계: 배송지와 포장 내역 확인



재고 7지표와 위치별 잔고

업무 규칙은 단위로, 격리·동시성은 실제 DB로, 흐름은 실제 브라우저로

npm run verify 한 명령: 타입 검사 → 단위 → 시뮬레이터 → PostgreSQL 통합 → 운영 빌드 → 브라우저

층	도구	건수	무엇을 검사하나
단위	Vitest	**371**	엔진 40개 명령의 상태 전이·수량 한도, 권한 판정, 화면 계산 로직
시뮬레이터	node:test	**55**	사건 생성·일정·독립 기대값 대조·복구 판정
통합	Vitest + 실제 PostgreSQL	**223**	RLS 회사 격리, 불변 트리거, 동시 요청·멱등, 요청 기록, 마이그레이션
브라우저	Playwright	**47**	스캔 수령→출하 전체 흐름, 응답 유실 후 복구, 권한·세션 만료, 페이지 넘김
합계		**696**	**2026-10-22 main(11e2095) 기준 전부 통과**

실패를 일부러 만든 시험

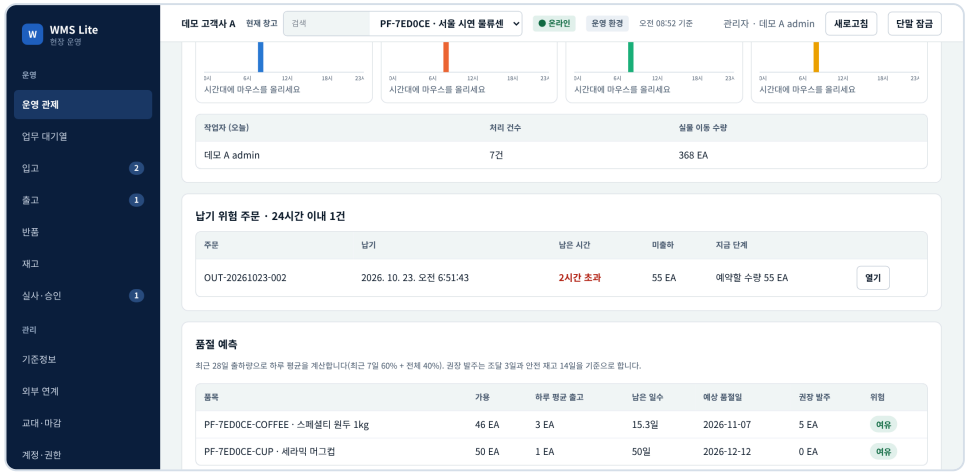
응답을 브라우저에서만 버리고 서버는 커밋시킨 뒤 재접속해 원래 요청을 복구하는지, 요청 기록 저장을 DB 트리거로 실패시켜도 업무 응답이 유지되는지 확인합니다.

원칙

GitHub Actions에 의존하지 않고 로컬에서 같은 명령으로 검증합니다. 테스트를 건너뛰거나 끄지 않습니다.

측정으로 확인한 개선

DB 앞에 왕복 20ms 지연을 넣어 같은 조건에서 비교: 기존 홈 화면 표시 1.47초→0.84초, 현장 운영 진입 2.64초→1.49초, 요청당 DB 왕복 22~34회→13~21회.



운영 관제의 품절 예측·배치도. 화면 수치와 성능 합격은 별개입니다.

해석 주의

696은 자동화 실행 사례 수이며 작성 분량이 아닙니다. 혼잡 시험의 목표 지연·완주 기준은 아직 통과하지 못했습니다.

AI가 구현하고, 사람이 규칙을 정하고 직접 써 보며 결함을 찾았습니다

개발 세션과 검증 세션을 나누고, 사용자가 결정하고 승인한 것만 main에 들어갑니다.

요구·규칙 결정 (사용자) → **구현 세션** (작업 브랜치에서 코드·시험 작성) → **검증 세션** (npm run verify · 전 화면 캡처 점검) → **사용자 확인** (직접 사용·결함 지적) → **"반영해"** (PR squash 병합 · main 일치 확인)

세션 분리

구현과 검증을 다른 세션·다른 도구(Codex, Claude Code)에 맡겨 같은 가정이 그대로 통과하지 않게 했습니다. 저장소 규칙(AGENTS.md·CLAUDE.md)에 로컬 검증 원칙과 출력 언어를 고정했습니다.

사용자가 정한 것

과납을 금지하지 않고 보류로 기록, 출고에 거래처·배송지 필수, 검색해서 추가하는 창고 선택, 서버 지역 등 업무·운영 판단은 사용자가 결정했습니다.

커밋 승인

AI는 작업 브랜치에만 푸시하고, 사용자가 "반영해"라고 해야 PR을 만들어 병합합니다. 병합 후 main이 검증한 커밋과 같은지 비교합니다.

직접 써 보며 찾은 결함

발견	원인과 조치
출고 확인창에 "도착: 없음"	창고 위치만 보여 줌 → 거래처·배송지 필수화, 포장 시점 주소를 스냅샷으로 보존
100개 예정에 105개 도착	전면 금지면 실물이 장부에서 사라짐 → 초과분 보류 기록 + 독립 승인
메뉴 이동 중 "권한 없음"	세션 만료를 권한 부족으로 처리 → 로그인 화면으로 보내고 브라우저 시험 추가
창고 100개를 체크박스로 나열	규모를 고려 안 함 → 검색해 추가하는 선택칸, 계정 20개·기준정보 50개씩 쪽 넘김
"로딩이 너무 느리다"	측정 결과 서버 미국 동부 기본값·요청당 DB 왕복 34회·렌더 차단 글꼴 → 지역 고정, 왕복 축소, 글꼴 자체 호스팅

AI에게 맡긴 일

코드·시험 작성, 문서 초안, 화면 캡처와 측정 스크립트 작성, 원인 후보 조사

사람이 한 일

업무 규칙과 예외 처리 결정, 직접 사용하며 결함 지적, 범위 결정, 병합 승인

판단 근거로 삼은 것

"통과했다"는 말이 아니라 로컬 전체 검증 결과, 변경 전후 측정값, 실제 화면 캡처

하지 않은 것을 구분하고, 근거 자료를 함께 제공합니다

하지 않은 것

항목	현재 상태
성능 합격	혼잡 시험 목표 미달 기록만 보존. 운영 환경(Vercel·Neon) 실측과 현장 규모 부하 검증은 하지 않았습니다.
장비 연동	실제 PDA·바코드 스캐너 SDK·라벨 프린터·택배 송장 API와 연결하지 않았습니다. 스캐너는 키보드 입력 방식으로만 지원합니다.
재고 관리 범위	LOT·유통기한·FEFO, 웨이브·배치 피킹은 범위에서 뺐습니다(상온 소모품 창고 기준).
오프라인 확정	재고 이중 반영 위험으로 온라인 우선 + 결과 확인 후 재시도 방식만 지원합니다.
실제 운영	합성 데이터와 시험 회사로만 검증했고, 실제 현장 운영 기록은 없습니다.

다음에 할 일

1. 혼잡 시험의 병목(전체 상태 읽기·잠금 대기)을 분리 계측하고 같은 부하로 재검증
2. 품목 선택칸을 서버 검색 방식으로 바꿔 품목 수가 늘어도 첫 화면이 느려지지 않게
3. PDA·라벨 프린터 SDK 연동과 택배 송장 API는 실제 장비·계약 조건을 받은 뒤 설계

저장소·문서

GitHub 저장소 ↗
github.com/jeonking80/WMS-SaaS-Platform

공개 데모 ↗
wms-saa-s-platform.vercel.app

시스템 구조 ↗
docs/학습자료/02-시스템-구조.md

핵심 설계 결정 ↗
문제 → 선택 → 대안 → 대가

DB 설계 ↗
docs/DB-설계.md

API 명세 ↗
docs/API-명세.md

구현·검증 추적표 ↗
요구사항별 구현 위치와 시험

사용자 매뉴얼 ↗
화면별 사용 방법

화면 캡처: 로컬 앱·실제 PostgreSQL·합성 데이터. 시뮬레이터 이미지는 기존 로컬 시험 캡처입니다. 코드 발췌는 커밋 ca8c389 기준입니다.